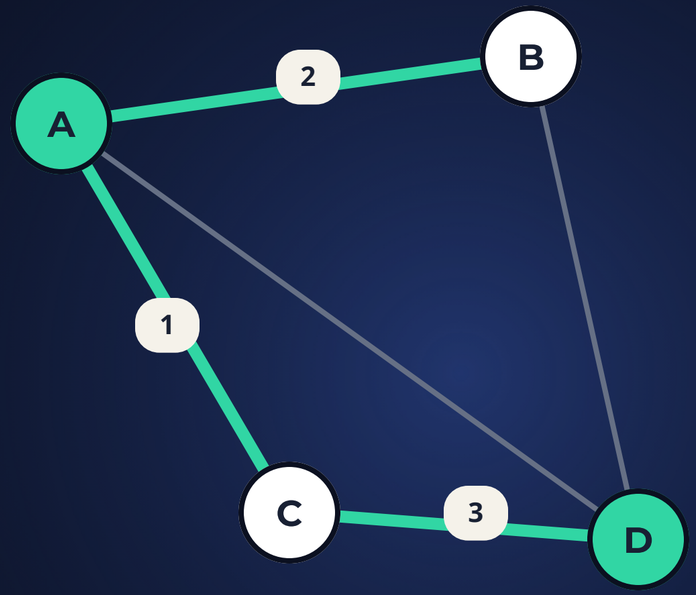
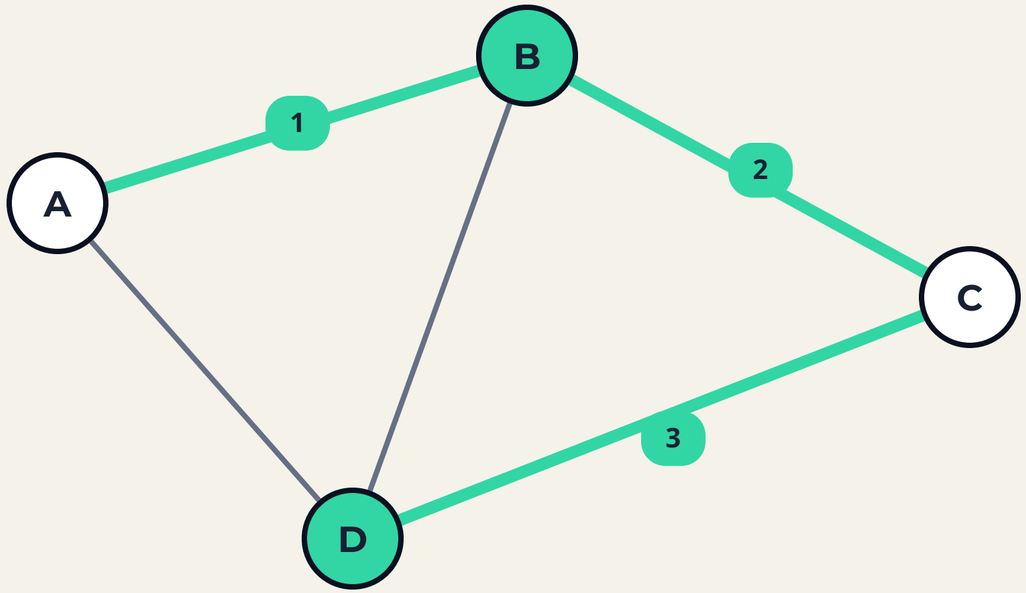
Kruskal's Algorithm: Building Minimum Spanning Trees

A clear guide with detailed explanation and worked example



A minimum spanning tree connects everything — for less.

An MST spans every vertex, contains no cycles, and minimizes the sum of selected edge weights.



Why it matters

Network design

Cost-saving infrastructure

Efficient connectivity

Greedy means: take the safest cheapest edge now.

At each step, choose the smallest-weight edge that does not create a cycle. Local choices accumulate into a globally optimal spanning tree.



Smallest edge weight 1

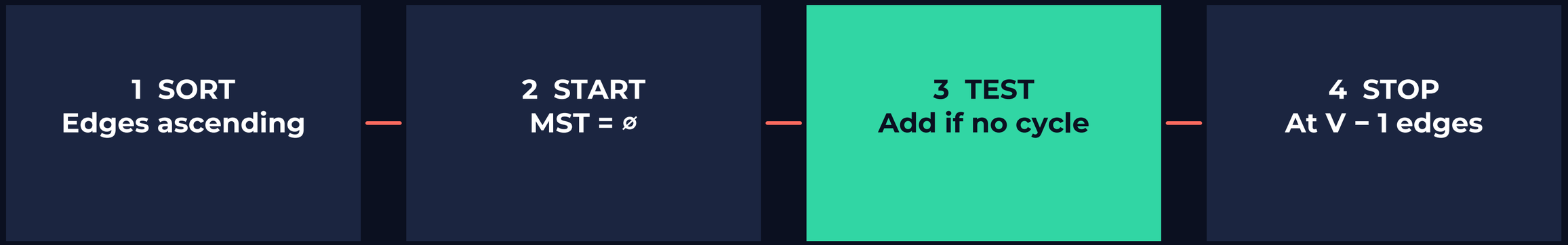


Next safe edge weight 2



Skip a cycle keep growing

Kruskal in four moves



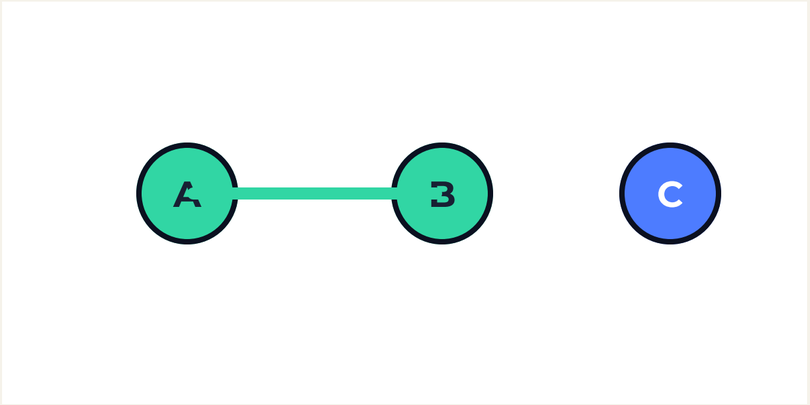

```
sort(E) → scan edges → union safe endpoints → stop at |V|−1
```

Union–Find answers one crucial question:

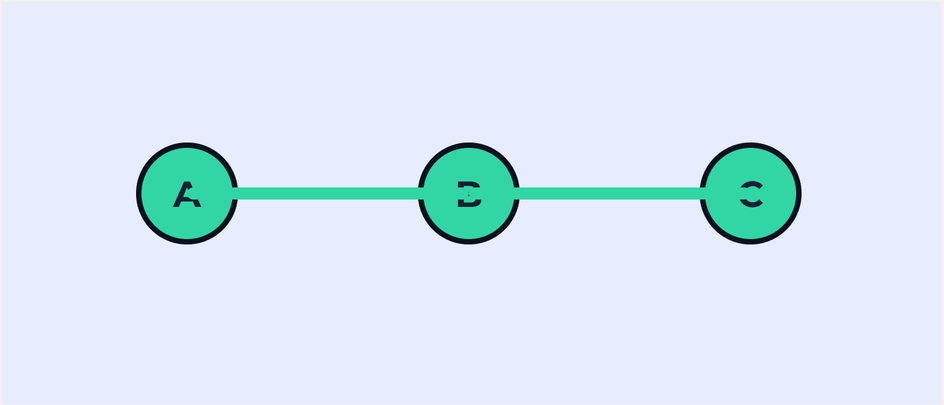
Are u and v already connected?

Find identifies a component representative. Union merges two components. If Find(u) = Find(v), adding the edge would close a cycle — so skip it.

BEFORE



AFTER UNION(B, C)



Kruskal’s Algorithm — pseudocode

```
KRUSKAL(G):
  MST ← ∅
  for each vertex v: MAKE-SET(v)
  sort edges E by ascending weight

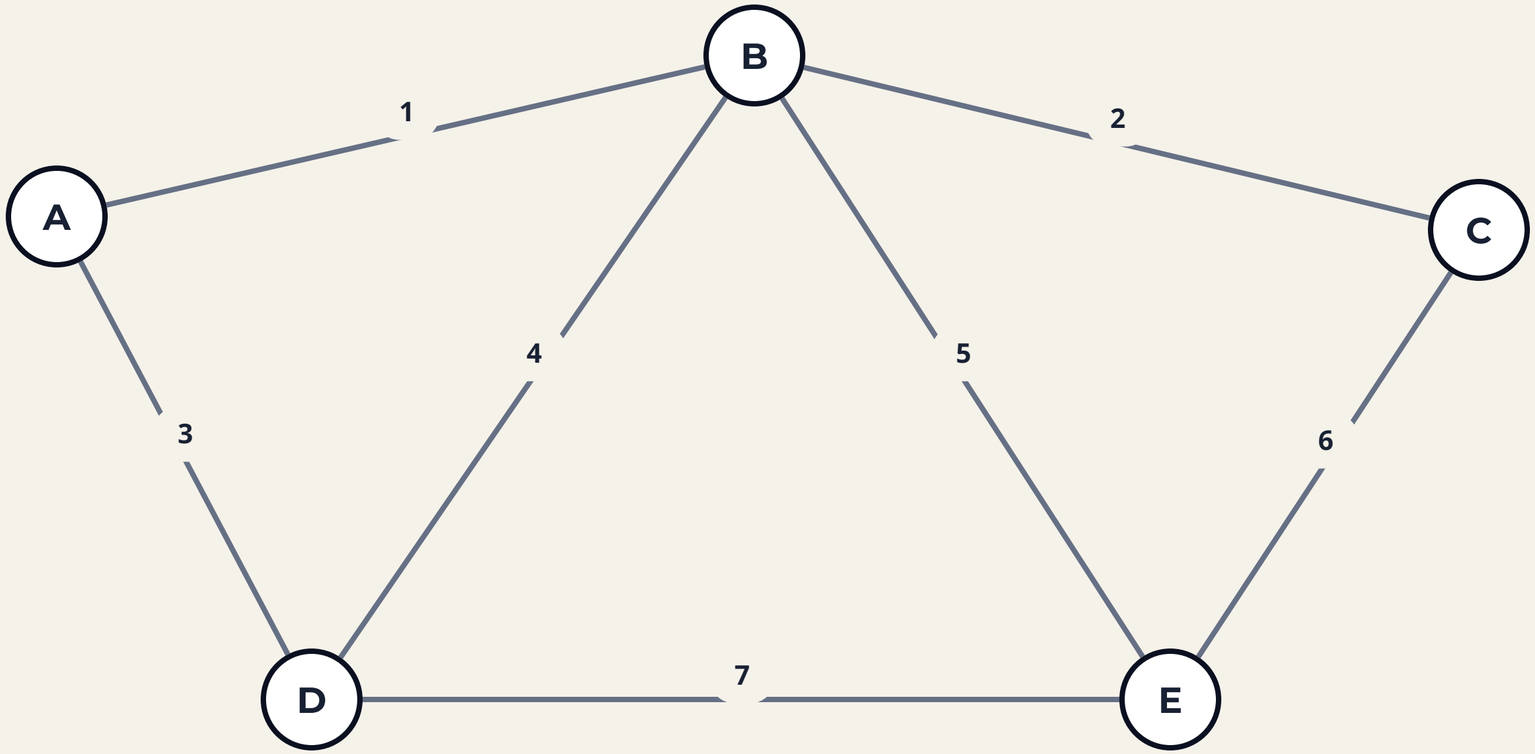
  for each edge (u, v, w) in E:
    if FIND(u) ≠ FIND(v):
      MST ← MST ∪ {(u, v, w)}
      UNION(u, v)
    if |MST| = |V| − 1: break

  return MST
```

The cycle test is the only gate.

Worked example: start with the full graph

Five vertices, seven weighted edges. Our target is 4 edges: $|V| - 1$.



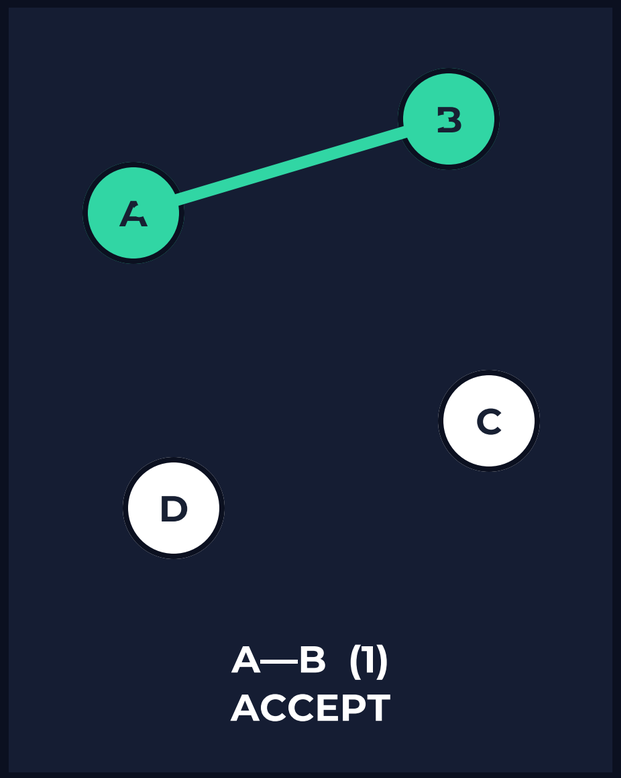
Sort first. Decisions become a simple scan.

The order is ascending by weight; equal weights may appear in any order.

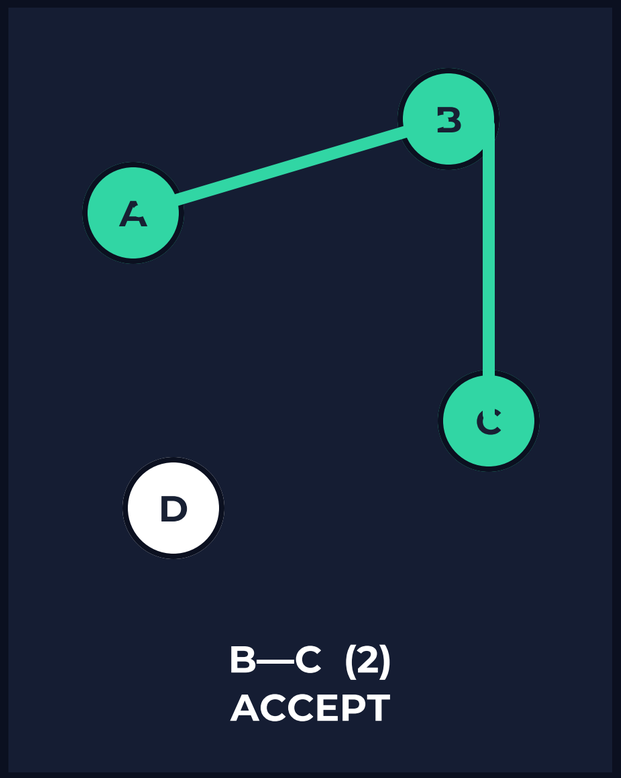
ORDER	EDGE	WEIGHT
1	A—B	1
2	B—C	2
3	A—D	3
4	B—D	4
5	B—E	5
6	C—E	6
7	D—E	7

Build the tree: accept, accept, accept.

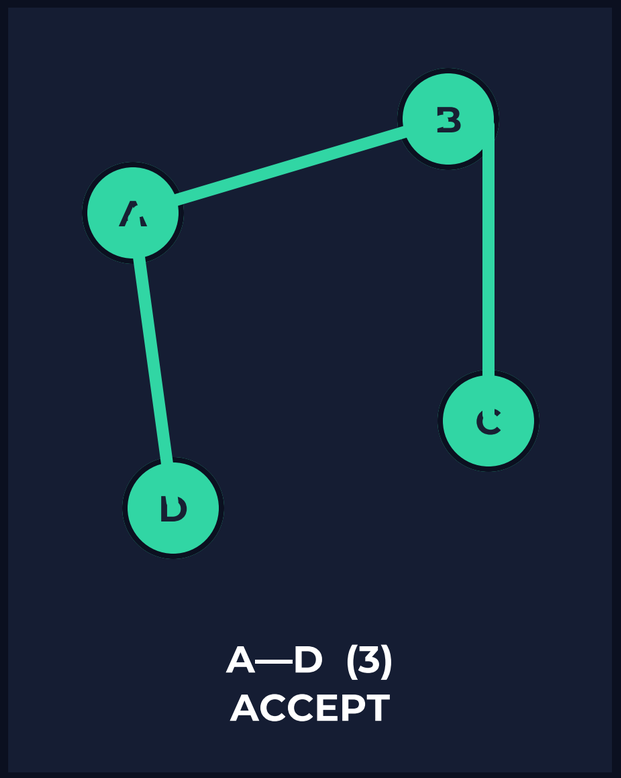
Each accepted edge joins two different components.



A—B (1)
ACCEPT



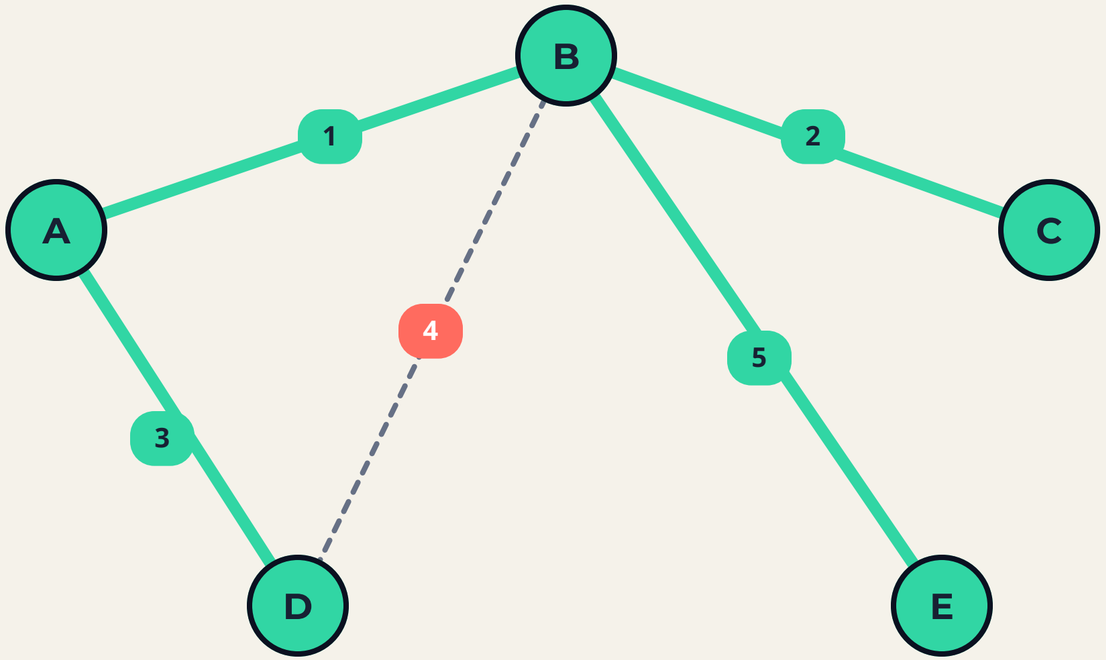
B—C (2)
ACCEPT



A—D (3)
ACCEPT

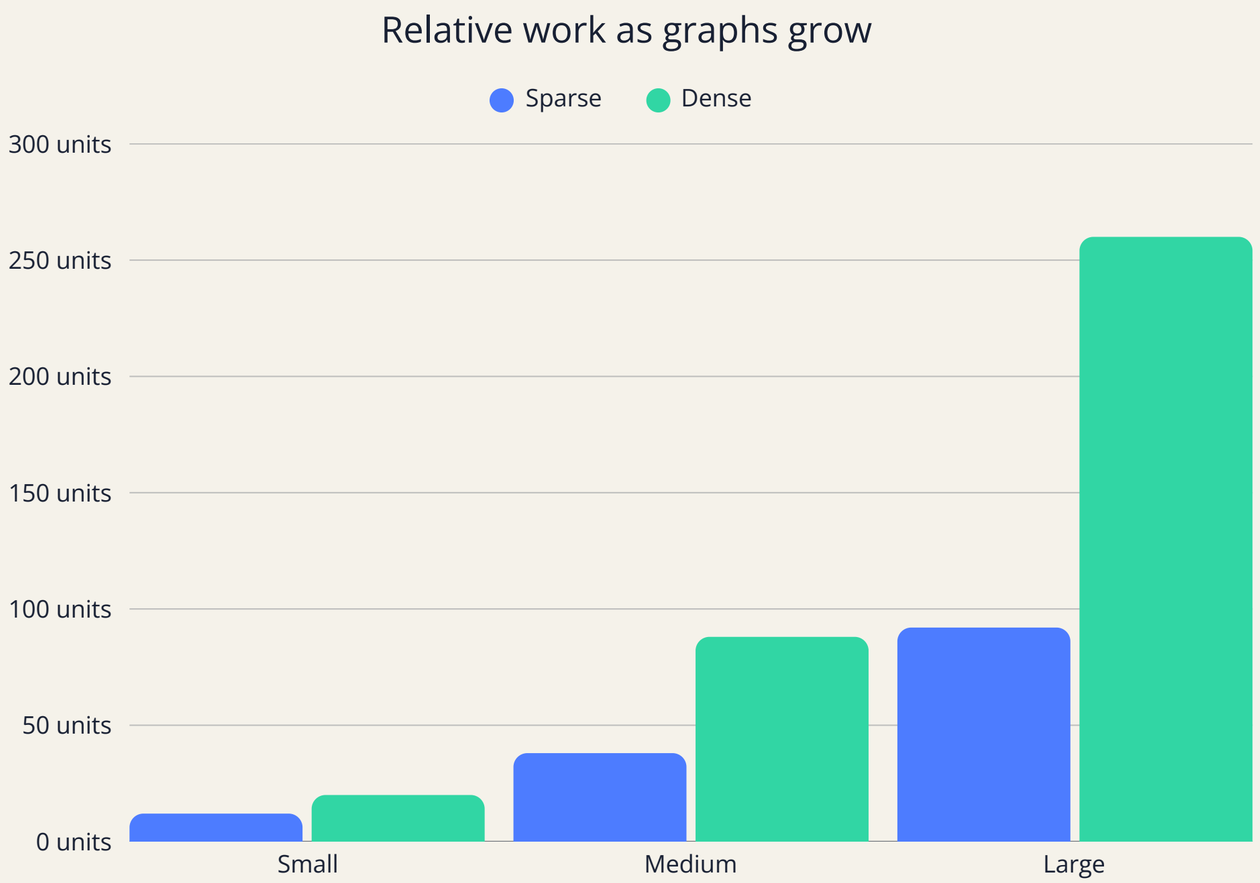
Skip the cycle. Add the last safe edge.

B—D (4) is rejected because B and D are already connected. B—E (5) is accepted, completing the MST.



TOTAL WEIGHT
1 + 2 + 3 + 5 = 11

Fast for sparse graphs. Useful in the real world.



O(E log E)

Sorting dominates; near-constant Union–Find operations keep cycle checks efficient.

APPLICATIONS

Network design

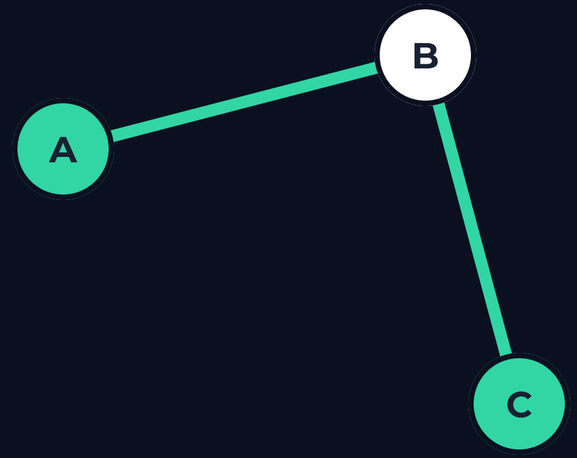
Clustering

Circuit layout

TRADE-OFF

Simple and excellent for sparse graphs; sorting can be costly on dense graphs.

Choose light. Avoid cycles. Connect all.



MST minimizes total connection cost.
Kruskal sorts edges, then makes greedy safe choices.
Union–Find makes cycle detection efficient.
The example MST has total weight 11.

Thank you

NAME :- AMISH DESHMUKH
SUBJECT :- DAA
PRN :- 25070521316
SECTION :- C

CREATED BY CANVA